

HDOC DAY : 33

Aim:

To write a menu-driven C program using functions to verify the associative property of matrix addition $(A+B)+C=A+(B+C)$ and matrix multiplication $(A \times B) \times C = A \times (B \times C)$.

Algorithm:

Main Function:

1. Display a menu with 3 options:
 - a. 1: Verify Associative Property of Addition
 - b. 2: Verify Associative Property of Multiplication
 - c. 3: Exit
2. Read the user's choice.
3. Call the selected function based on the choice.
4. Repeat the process using a do-while loop until the user selects option 3.

Option 1: Addition Associativity Verification

1. Read matrix dimensions m and n .
2. Input values for matrices A , B , and C of size $m \times n$.
3. Calculate LHS: $L1=A+B$, then $LHS=L1+C$.
4. Calculate RHS: $R1=B+C$, then $RHS=A+R1$.
5. Display matrix LHS and matrix RHS.

6. Compare elements of LHS and RHS. If all corresponding elements are equal, print "Associative property of addition verified." Else, print "Not verified."

Option 2: Multiplication Associativity Verification

1. Read dimensions m,n,p,q for matrices A(m×n), B(n×p), and C(p×q).
2. Input values for matrices A, B, and C.
3. Calculate LHS: L1=A×B (size m×p), then LHS=L1×C (size m×q).
4. Calculate RHS: R1=B×C (size n×q), then RHS=A×R1 (size m×q).
5. Display matrix LHS and matrix RHS.
6. Compare elements of LHS and RHS. If all elements match, print "Associative property of multiplication verified." Else, print "Not verified."

C code :

```
#include <stdio.h>
```

```
void readMat(int r, int c, int mat[10][10]) {
```

```
    int i, j;
```

```
    for (i = 0; i < r; i++) {
```

```
        for (j = 0; j < c; j++) {
```

```
            scanf("%d", &mat[i][j]);
```

```
        }
```

```
    }
```

```
}
```

```
void printMat(int r, int c, int mat[10][10]) {
```

```
    int i, j;
```

```
    for (i = 0; i < r; i++) {
```

```
        for (j = 0; j < c; j++) {
```

```
            printf("%d ", mat[i][j]);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
}
```

```
void addMat(int r, int c, int A[10][10], int B[10][10], int res[10][10]) {
```

```
    int i, j;
```

```
    for (i = 0; i < r; i++) {
```

```
        for (j = 0; j < c; j++) {
```

```
            res[i][j] = A[i][j] + B[i][j];
```

```
        }
```

```
    }
```

```
}
```

```
void mulMat(int r1, int c1, int A[10][10], int r2, int c2, int B[10][10], int
res[10][10]) {

    int i, j, k;

    for (i = 0; i < r1; i++) {

        for (j = 0; j < c2; j++) {

            res[i][j] = 0;

            for (k = 0; k < c1; k++) {

                res[i][j] += A[i][k] * B[k][j];

            }

        }

    }

}
```

```
int checkEqual(int r, int c, int A[10][10], int B[10][10]) {

    int i, j;

    for (i = 0; i < r; i++) {

        for (j = 0; j < c; j++) {

            if (A[i][j] != B[i][j]) {

                return 0;

            }

        }

    }

}
```

```
}  
  
return 1;  
  
}  
  
void doAddition() {  
  
    int m, n;  
  
    int A[10][10], B[10][10], C[10][10];  
  
    int temp[10][10], LHS[10][10], RHS[10][10];  
  
    printf("\nEnter m and n: ");  
  
    scanf("%d %d", &m, &n);  
  
    printf("Enter A:\n");  
  
    readMat(m, n, A);  
  
    printf("Enter B:\n");  
  
    readMat(m, n, B);  
  
    printf("Enter C:\n");  
  
    readMat(m, n, C);
```

```
addMat(m, n, A, B, temp);
```

```
addMat(m, n, temp, C, LHS);
```

```
addMat(m, n, B, C, temp);
```

```
addMat(m, n, A, temp, RHS);
```

```
printf("\nLHS Matrix:\n");
```

```
printMat(m, n, LHS);
```

```
printf("RHS Matrix:\n");
```

```
printMat(m, n, RHS);
```

```
if (checkEqual(m, n, LHS, RHS)) {
```

```
    printf("Addition is Associative!\n");
```

```
} else {
```

```
    printf("Not Associative!\n");
```

```
}
```

```
}
```

```
void doMultiplication() {
```

```
    int m, n, p, q;
```

```
int A[10][10], B[10][10], C[10][10];
```

```
int temp1[10][10], temp2[10][10], LHS[10][10], RHS[10][10];
```

```
printf("\nEnter m, n, p, q: ");
```

```
scanf("%d %d %d %d", &m, &n, &p, &q);
```

```
printf("Enter A:\n");
```

```
readMat(m, n, A);
```

```
printf("Enter B:\n");
```

```
readMat(n, p, B);
```

```
printf("Enter C:\n");
```

```
readMat(p, q, C);
```

```
mulMat(m, n, A, n, p, B, temp1);
```

```
mulMat(m, p, temp1, p, q, C, LHS);
```

```
mulMat(n, p, B, p, q, C, temp2);
```

```
mulMat(m, n, A, n, q, temp2, RHS);
```

```
printf("\nLHS Matrix:\n");
```

```
printMat(m, q, LHS);
```

```
printf("RHS Matrix:\n");
```

```
printMat(m, q, RHS);
```

```
if (checkEqual(m, q, LHS, RHS)) {
```

```
    printf("Multiplication is Associative!\n");
```

```
} else {
```

```
    printf("Not Associative!\n");
```

```
}
```

```
}
```

```
int main() {
```

```
    int choice;
```

```
    do {
```

```
        printf("\n1. Addition Test\n");
```

```
        printf("2. Multiplication Test\n");
```

```
        printf("3. Exit\n");
```

```
        printf("Enter choice: ");
```



```
scanf("%d", &choice);

if (choice == 1) {
    doAddition();
} else if (choice == 2) {
    doMultiplication();
} else if (choice == 3) {
    printf("Exit\n");
} else {
    printf("Wrong choice!\n");
}

} while (choice != 3);

return 0;

}
```

OUTPUT SCREENSHOT :

```
[manalithakar@Unknown_0a:57:f1:e7:bf:b0 ~ % nano 33.c
[manalithakar@Unknown_0a:57:f1:e7:bf:b0 ~ % gcc 33.c -o 33
[manalithakar@Unknown_0a:57:f1:e7:bf:b0 ~ % ./33
```

```
1. Addition Test
2. Multiplication Test
3. Exit
```

Enter choice: 1

Enter m and n: 2 2

Enter A:

1 2

3 4

Enter B:

5 1

2 3

Enter C:

0 4

1 2

LHS Matrix:

6 7

6 9

RHS Matrix:

6 7

6 9

Addition is Associative!

```
1. Addition Test
2. Multiplication Test
3. Exit
```

Enter choice: 2

Enter m, n, p, q: 2 2 2 2

Enter A:

1 2

3 0

Enter B:

2 1

1 4

Enter C:

3 0

1 2

LHS Matrix:

21 18

21 6

RHS Matrix:

21 18

21 6

Multiplication is Associative!

```
1. Addition Test
2. Multiplication Test
3. Exit
```

Enter choice: 3

Exit

manalithakar@Unknown_0a:57:f1:e7:bf:b0 ~ %